

Production-Grade AI CI Agent

Architecture, Reuse Strategy, Security Model, and Build Plan

A fresh-start design for an AI agent that understands repositories, selects compatible CI capabilities, generates pipelines, validates them, and applies governance.

Production-Grade AI CI Agent - High-Level Architecture

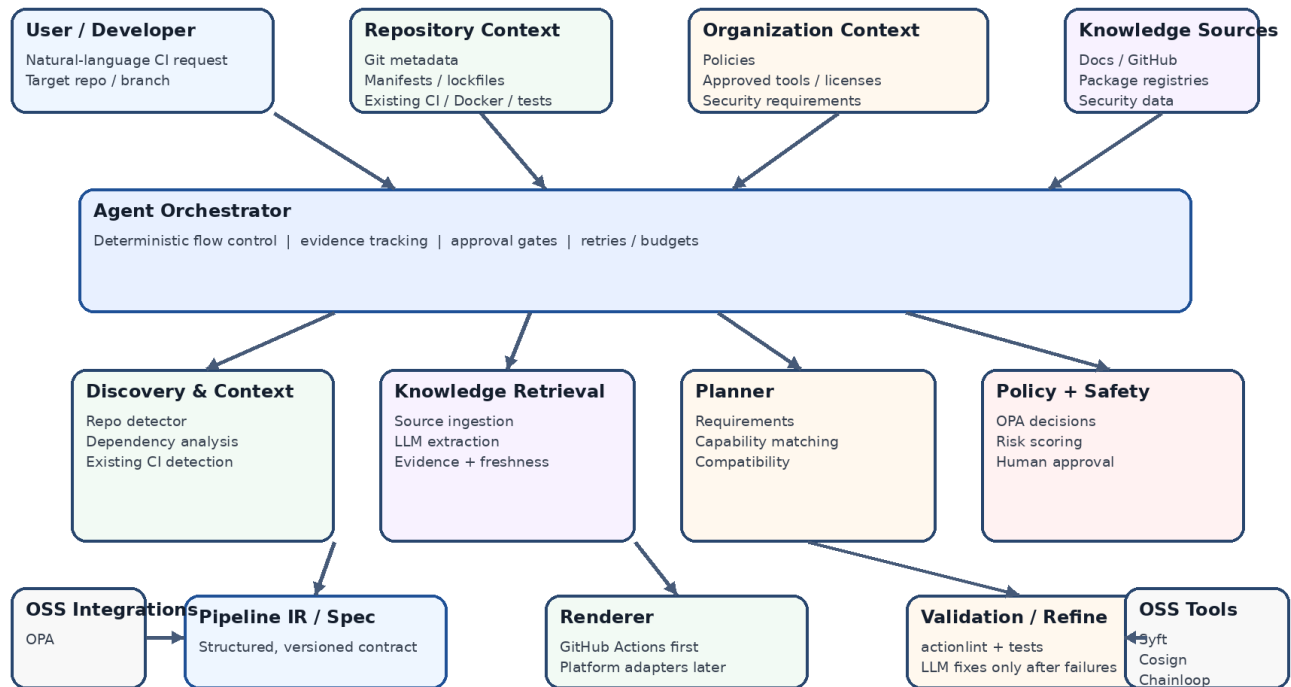


Figure 1. Recommended production architecture. The AI reasons over evidence; deterministic services and established OSS enforce safety and correctness.

1. Executive Summary

The goal is not to build a better YAML template generator. The goal is to build an AI CI agent that can look at a repository, understand what the project needs, use current knowledge from trusted sources, apply organization policy, create a pipeline plan, generate CI configuration, validate it, explain the result, and stop for human approval when risk requires it.

- Start from a clean project rather than extending the current prototype. Keep the prototype as a reference and copy only proven concepts.
- Do not hardcode a large tool registry. Build a knowledge ingestion layer that can discover tools and capabilities from trusted sources and normalize the findings into our schema.
- Let the LLM reason and plan, but do not let it be the final authority for policy, permissions, validation, or raw command execution.

- Use established open-source projects where they already solve the problem: OPA for policy, actionlint for GitHub workflow validation, Syft for SBOMs, Cosign for signing/attestations, and Chainloop for supply-chain evidence when that phase is reached.
- Keep a structured intermediate representation (PipelineSpec / Pipeline IR) between AI planning and YAML rendering. This is one of the most important design decisions because it makes the system testable, portable, and safer.

2. What the Agent Must Eventually Do

The supplied Phase A/B flow is treated as the target capability map. The first release should not implement every item at once, but the architecture should leave room for them.

Target area	Simple meaning	Architectural owner
Trigger validation	Make sure the event, source revision, pull request and branch context are real and traceable.	GitHub integration + deterministic checks
Checkout / baseline	Resolve the exact revision and inspect the repository safely.	Repository adapter + sandbox
Format / lint	Choose stack-appropriate formatting and lint checks.	Knowledge + planner + generated workflow
SAST / code quality	Run security/static analysis using approved tools.	Tool knowledge + policy + CI
Unit / architecture tests	Run tests in the correct order and environment.	Repo discovery + planner
Secret / SCA	Detect secrets and vulnerable dependencies.	Specialized OSS scanners / APIs
Policy gate	Make deterministic allow/deny/approval decisions.	OPA + policy bundles
Human approval	Require review for high-risk changes.	Agent governance layer
SBOM / artifact security	Produce, scan, sign and record build evidence.	Syft + scanners + Cosign + Chainloop
Merge decision	Produce a decision and evidence summary that a reviewer can understand.	Agent + evidence store + SCM adapter

3. Design Principles

AI is the planner, not the enforcement engine: The model may recommend a tool or step, but OPA, deterministic validators, permissions, and approval gates decide what is actually allowed.

Evidence before decisions: Important facts should carry their source: repository file, official documentation page, package metadata, policy rule, or tool output.

Structured data between reasoning and execution: Use typed JSON/IR objects instead of asking the model to directly write final YAML or shell scripts.

Reuse before reinvent: Prefer existing OSS packages, CLIs, SDKs, and services when they are mature and fit the requirement.

Fail closed: Unknown tool, missing policy, validation failure, unsafe command, or insufficient approval should stop the operation rather than silently continue.

Minimal blast radius: The agent should not get broad shell or production credentials. Every external action should pass through a controlled tool boundary.

Reproducibility: Pin dependency versions, record source revisions, record model/version, and make pipeline generation deterministic where possible.

Human approval for high-risk changes: Generation can be automated; committing or changing protected CI/security behavior should be approval-aware.

4. Fresh-Start Architecture

I recommend a clean repository with a modular monolith first. It should have clear internal boundaries but should not be split into many microservices on day one. Once usage and scaling justify it, the knowledge service, policy service, execution workers, or web API can be separated.

Agent Orchestrator — Owns the workflow: inspect → retrieve → plan → policy-check → generate → validate → refine → approve → publish.

Repository Intelligence — Reads manifests, lockfiles, existing CI, Dockerfiles, test layout, build files and Git metadata. It creates evidence-backed context, not guesses.

Knowledge System — Ingests trusted external sources and turns them into normalized tool/capability/platform records. Includes source, timestamp, version and evidence.

Planner — Maps user intent + repo facts + organization constraints into a pipeline plan. It never bypasses policy.

Pipeline IR — Versioned internal representation for stages, jobs, capabilities, dependencies, permissions, artifacts, gates and evidence requirements.

Renderers — Platform-specific deterministic renderers. Start with GitHub Actions. Add Azure Pipelines/Jenkins/GitLab later behind a common interface.

Validation — Syntax/semantic checking, policy checks, security checks, generated workflow tests, and optionally a sandboxed dry run.

Governance / Approval — Risk classification, human approval, immutable decision records and audit trail.

Integrations — Thin adapters around GitHub, OPA, actionlint, Syft, Cosign, Chainloop, vulnerability sources, package registries and future SCM/CI targets.

Observability — Metrics, structured logs, traces, cost tracking, model latency, tool failures, policy denials and pipeline validation history.

5. The Agent Loop in Simple Words

Step 1 - Receive a request: “Create CI for this repository and include lint, tests and security scanning.”

Step 2 - Identify the exact repository and revision. Never assume the default branch is the only source of truth.

Step 3 - Inspect the repository using deterministic discovery tools. Find runtimes, package managers, dependency files, test commands, Docker, and existing CI.

Step 4 - Retrieve relevant knowledge from trusted sources. Example: if a project uses Python, look up current compatible lint, test, SAST and packaging information rather than relying on a stale hardcoded registry.

Step 5 - Ask the LLM to produce structured intent and a candidate plan. Use Structured Outputs / JSON Schema so the model response is machine-checkable. Microsoft Foundry supports structured outputs and function calling in its Responses API. (Microsoft Foundry Responses API documentation)

Step 6 - Send the plan to deterministic policy evaluation. OPA becomes the policy decision point rather than a hand-written Python allow/deny list. OPA is a general-purpose policy engine designed for context-aware enforcement. (Open Policy Agent project documentation)

Step 7 - Convert the approved plan into Pipeline IR. This step is deterministic and testable.

Step 8 - Render GitHub Actions YAML from Pipeline IR. The renderer should be deterministic and should not need an LLM to format YAML.

Step 9 - Validate with actionlint plus our own higher-level checks. actionlint is purpose-built for static checking GitHub Actions workflows and is MIT licensed. (actionlint project documentation)

Step 10 - If validation fails, feed the exact failure evidence back to the planner/refiner. The model may propose a fix, but the result must pass the same gates again.

Step 11 - For high-risk changes, stop for approval before committing or opening a pull request.

Step 12 - Record the inputs, plan, policy result, generated artifact hash, validation outputs, approval, and final decision as evidence.

6. Dynamic Knowledge Instead of a Hardcoded Tool Registry

This is one of the biggest changes from the current prototype. A giant Python list of tools should not become the long-term source of truth.

6.1 Trusted knowledge sources

- Official project documentation
- Official GitHub repositories and release metadata
- Package registries and lockfile metadata
- Official GitHub Action metadata / marketplace information where available
- Official security advisories and vulnerability APIs
- Organization-owned policy and approved-tool catalogs
- Optional secondary sources only when a primary source is unavailable or for corroboration

6.2 Ingestion flow

Step 1 - Fetcher retrieves the source and records URL, revision, retrieval time and source type.

Step 2 - Content is treated as untrusted data. Repository text or web text never becomes agent instructions.

Step 3 - LLM extracts structured facts such as capability, language, installation method, CI integration, supported versions and commands.

Step 4 - Schema validation rejects malformed records.

Step 5 - Rule-based checks compare extracted claims against source text where possible.

Step 6 - Each fact keeps evidence and freshness metadata so the planner can prefer current, authoritative information.

Step 7 - Deduplication merges multiple sources into one canonical record while retaining all supporting evidence.

```
{
  "tool": "<canonical-name>",
  "capabilities": ["lint"],
  "ecosystems": ["python"],
  "install_methods": [{"type": "pip", "command": "pip install <tool>"}],
  "ci_integrations": ["github-actions"],
  "versions": ["..."],
  "source": {
    "url": "...",
    "retrieved_at": "...",
    "evidence": ["..."]
  }
}
```

7. LLM Boundary and Safety Model

The model should be powerful but boxed in. It should reason over structured inputs, call approved tools, and return structured objects. It should not receive raw production credentials or an unrestricted shell.

Structured Outputs: Use JSON Schema/Structured Outputs for requirements, plan objects, knowledge records and repair proposals. Microsoft Foundry documents Structured Outputs as the schema-constrained option; plain JSON mode only guarantees valid JSON, not schema conformance. (Microsoft Foundry Structured Outputs documentation)

Tool calling: Expose narrow functions such as `scan_repository`, `retrieve_knowledge`, `evaluate_policy`, `render_workflow`, `validate_workflow`, and `open_pull_request`. The model chooses a tool; the tool enforces its own validation.

Untrusted content: README files, source code comments, issue text and web pages may contain prompt injection. Treat all retrieved text as data. Microsoft Foundry provides guardrails for prompt-injection-related risks, but application-level trust boundaries are still required. (Microsoft Foundry security/guardrails documentation)

Execution isolation: Use a sandbox/worker for any command execution or dry-run. Use allowlisted commands, timeouts, resource budgets, network controls and read-only credentials by default.

No secret exfiltration: Never send secrets into the LLM context. Redact logs and tool output. Use short-lived identity tokens and secret stores for actual CI integrations.

Approval boundary: The agent can prepare changes without permission to merge or modify protected resources. Approval must be explicit and recorded.

8. Open-Source Reuse Strategy

The principle is simple: use our engineering effort on the intelligence and orchestration that differentiates the project. Wrap or invoke mature OSS for specialized capabilities.

Project	Role in our architecture	Reuse level	Why	License / note
talkops-ai/ci-copilot	Primary architectural reference for CI agent flow, context detection, policy validation, rendering, actionlint refinement, HITL and PR flow.	Reference / selective code reuse	Closest to the problem we are solving.	Apache-2.0; review file-level license before copying code. (CI-Copilot repository)
SanjaySundarMurthy/pipeline-generator	Reference and possible renderer ideas.	Reference / selective components	Useful deterministic spec-to-platform generation.	Repository describes multi-platform generation and auto-detection. (pipeline-generator repository)
open-devops-agent/open-devops-agent	Security and agent-operation reference.	Reference / patterns	Grounded AI, plugins, audit, guarded execution and rollout ideas.	MIT; repository documents security and operational practices. (open-devops-agent repository)
skundu42/opendevops	Execution safety model.	Reference / patterns	Default-deny policy, narrow argv execution, credential boundary and audit chain.	Useful security pattern for our executor. (opendevops repository)
stakpak/agent	Agent core/tool boundary ideas.	Reference / selective components	Useful for keeping production systems behind controlled tool interfaces.	Apache-2.0 per repository metadata. (Stakpak agent repository)
JenkinsPipelineUnit	Jenkins workflow tests.	Direct integration later	Avoid implementing Jenkins pipeline unit tests ourselves.	MIT. (JenkinsPipelineUnit repository)
OPA	Policy decision	Direct	Use Rego and OPA	CNCF graduated

	point.	integration	rather than growing a custom policy engine.	project. (Open Policy Agent project documentation)
Syft	SBOM generation.	Direct integration	Mature SBOM CLI/library; supports many ecosystems and CycloneDX/SPDX.	Apache-2.0. (Syft repository)
Cosign	Artifact signing and attestation.	Direct integration	Use established Sigstore tooling instead of implementing signing.	Sigstore project; supports signing and attestations. (Cosign / Sigstore documentation)
Chainloop	Evidence store / supply-chain evidence.	Direct integration later	Good fit for the evidence half of Phase B.	Open-source evidence store with control plane, CAS and CLI. (Chainloop repository)
actionlint	GitHub Actions validation.	Direct integration	Purpose-built workflow static checker.	MIT. (actionlint project documentation)

9. Recommended Build Phases

Phase 0 - Foundation — Fresh repository; configuration; Azure Foundry client; typed models; logging; tests; GitHub adapter skeleton; secret handling; CI for the agent itself.

Phase 1 - Real CI Agent — GitHub Actions only. Repository context detection, dynamic knowledge retrieval, LLM intent parsing, Pipeline IR, deterministic renderer, actionlint validation, explanation and dry-run output.

Phase 2 - Governance — OPA policy bundles; approved actions/tools; license policy; risk classification; approval gates; immutable decision records.

Phase 3 - Security Pipeline — Secret scanning, SCA, SAST and container scanning through integrated OSS tools. No custom scanners unless a real gap appears.

Phase 4 - Supply Chain — SBOM with Syft, signing/attestation with Cosign, evidence and provenance with Chainloop or equivalent, artifact digests, release evidence.

Phase 5 - PR Automation — Branch creation, workflow commit, PR creation, required checks, reviewer summary and merge recommendation. Human approval remains explicit.

Phase 6 - Execution / Feedback — Sandboxed dry-run, optional CI execution observation, failure diagnosis, bounded repair loop, revalidation, and eventually controlled auto-fix for low-risk cases.

Phase 7 - Multi-platform — Add Azure Pipelines/Jenkins/GitLab behind platform adapters. Reuse JenkinsPipelineUnit for Jenkins testing where applicable. (JenkinsPipelineUnit repository)

10. Production-Grade Requirements

Identity: Use Microsoft Entra identity and RBAC in Azure-hosted deployments. Foundry Agent Service supports dedicated agent identity, RBAC and private networking; even if we keep our own orchestrator, these controls are a strong reference for the hosting model. (Microsoft Foundry Agent Service documentation)

Network isolation: Prefer private networking for internal repositories and policy/knowledge stores. Restrict outbound destinations for tools and fetchers where practical.

Secrets: Use managed identity / secret store for runtime integrations. Never place long-lived GitHub tokens in prompts or source code.

GitHub permissions: Generated workflows should default to minimum permissions. When cloud access is needed, prefer OIDC with narrowly scoped trust rather than stored long-lived credentials. GitHub documents that ``id-token: write`` is specifically needed to request the OIDC token and does not itself grant write access to other resources. (GitHub Actions OIDC documentation)

Audit: Persist request, repo revision, context hash, knowledge sources, model name/version, plan hash, policy result, generated workflow hash, validator output, approval identity and final action.

Observability: Expose latency, token usage/cost, tool-call counts, policy denials, validation errors, refinement attempts, approval wait time, generation success rate and drift in tool knowledge.

Reliability: Use timeouts, retries only for safe/idempotent operations, bounded model/tool calls, circuit breakers for external APIs, and deterministic fallbacks.

Data governance: Define retention for source content, prompts, generated artifacts and evidence. Redact secrets and sensitive data from logs. Foundry documentation describes how prompts, uploaded data and tool outputs may be processed depending on the configuration, so deployment choices must be made explicitly. (Microsoft Foundry data/privacy documentation)

Release management: Version the agent, prompts, schemas, policy bundles, renderer templates and knowledge schema. Treat each as a release artifact.

11. Core Data Contracts

The following objects should be stable contracts between modules. The LLM should fill structured fields, but deterministic code validates them.

RepositoryContext — Repo URL, commit SHA, language/runtime, package manager, dependency files, tests, Docker, existing CI, security tooling, evidence list.

KnowledgeRecord — Canonical tool/capability/platform identity, supported ecosystems, install methods, versions, CI integrations, source URLs, retrieval time, evidence and confidence.

UserIntent — Requested platform, capabilities, deployment target, constraints, prohibited tools, required controls and requested trigger behavior.

PolicyDecision — ALLOW / DENY / APPROVAL_REQUIRED plus rule IDs, reasons and evidence.

PipelineIR — Stages/jobs/steps, capability, tool reference, dependencies, permissions, environment, artifacts, gates, validation requirements.

ValidationResult — Validator name/version, pass/fail, errors/warnings, source file, remediation hint.

DecisionEvidence — Immutable record of what was known, what was decided, which policy applied, what was generated, and who approved it.

12. Recommended Repository Layout

```
ci-agent/
├── agent/           # orchestration and decision flow
├── api/             # REST/A2A/MCP-facing interfaces later
├── discovery/       # repository and runtime intelligence
├── knowledge/       # source ingestion, extraction, retrieval, schemas
├── planner/         # intent -> plan -> IR
├── pipeline/        # IR models and deterministic renderers
├── policy/          # OPA adapter + Rego bundles
├── validation/      # actionlint, schema, security checks
├── integrations/    # GitHub, Syft, Cosign, Chainloop, scanners
├── execution/       # sandboxed command/tool runner
├── evidence/        # evidence model and persistence
├── security/        # auth, secret redaction, trust boundaries
├── observability/   # logs, metrics, tracing, cost
├── tests/           # unit, integration, golden, security, evals
├── config/          # environment/config templates
└── docs/            # ADRs, threat model, runbooks, design docs
```

13. Testing and Evaluation Strategy

Unit tests — Pure functions: parsing, IR conversion, policy adapters, renderers, validators and evidence generation.

Golden tests — Given a fixed repo snapshot + fixed intent + fixed knowledge snapshot, expect the same Pipeline IR and stable YAML.

Integration tests — Real OPA evaluation, actionlint, GitHub API mocks, knowledge ingestion, Syft/Cosign invocation wrappers.

Security tests — Prompt injection fixtures in README/issues/docs; path traversal; malicious filenames; shell injection; credential leakage; permission escalation; unsafe tool requests.

Agent evaluations — A dataset of representative repositories and requests with expected decisions. Measure intent accuracy, unsupported-claim rate, policy violations, invalid workflow rate and repair success.

Drift tests — Re-run the same test set when model, knowledge schema, policies or renderer versions change.

14. What We Should Not Build Ourselves

- A custom SBOM engine.
- A custom artifact-signing system.
- A custom general-purpose policy language.

- A custom GitHub Actions parser/linter if actionlint covers the need.
- A giant handwritten tool catalog as the primary source of truth.
- A general-purpose shell with unrestricted LLM access.
- A microservice-heavy deployment before the usage pattern requires it.
- A multi-agent swarm simply because the system is called an AI agent.

15. Main Risks and How the Architecture Handles Them

Hallucinated tools/commands: Ground the plan in retrieved knowledge and repository evidence; store source URLs; validate generated commands.

Stale knowledge: Store retrieval timestamps, versions and source revisions; refresh important tool records periodically.

Prompt injection: Treat repository/web text as untrusted input; separate instructions from data; keep tool permissions outside the model. Foundry provides additional guardrails, but they do not replace application-level controls. (Microsoft Foundry security/guardrails documentation)

Over-permissioned workflows: Generate least-privilege GitHub permissions and use OIDC for cloud authentication when required. (GitHub Actions OIDC documentation)

Bad auto-fix: Only auto-apply low-risk deterministic changes; require approval for protected branches, policy changes, security control removal and production deployment behavior.

Supply-chain compromise: Pin action/tool versions, validate digests where appropriate, generate SBOMs, sign artifacts, and preserve evidence.

Tool outage: Keep adapters narrow, cache non-sensitive knowledge, make tool failures explicit, and avoid silent fallback to unsafe alternatives.

16. Recommended First Production-Grade Milestone

The first milestone should be intentionally small but architecturally real. I recommend the following definition of done:

- Input: repository + natural-language CI request.
- Platform: GitHub Actions only.
- Knowledge: at least a few trusted official sources and a normalized schema; no hardcoded master tool registry.
- Discovery: runtime, package manager, dependency files, tests, Docker and existing CI.
- Planning: LLM produces UserIntent and PipelineIR proposal using Structured Outputs.
- Policy: OPA evaluates allowed capabilities/tools/actions and approval requirements.
- Rendering: deterministic GitHub Actions renderer.
- Validation: actionlint + schema checks + basic security checks.
- Governance: human approval before commit/PR.
- Evidence: every important decision is traceable to repo evidence, knowledge source or policy rule.
- Safety: no unrestricted shell, no secret exposure to the model, no direct production mutation.
- Testing: unit + golden + integration + prompt-injection/security tests.

17. Example End-to-End Decision

Suppose a user says: “Create GitHub Actions CI for this Python API. Run tests, linting and security checks. Do not use Semgrep.”

Step 1 - Repository discovery finds Python, Python version constraints, package manager and test configuration.

Step 2 - Knowledge retrieval finds the current approved options for the requested capabilities and records evidence.

Step 3 - The LLM outputs structured intent: GitHub, test + lint + SAST, Semgrep prohibited.

Step 4 - The planner selects only tools that are compatible with the repository and organization policy.

Step 5 - OPA checks the candidate plan and returns a decision.

Step 6 - Pipeline IR records jobs, steps, permissions, dependencies and required gates.

Step 7 - The renderer creates the workflow.

Step 8 - actionlint and policy checks run. If validation fails, the agent gets the exact errors and proposes a bounded correction.

Step 9 - After the workflow passes, the user reviews the plan and final YAML.

Step 10 - Only after approval does the GitHub adapter create the branch/commit/PR.

Step 11 - The agent records the decision and evidence so a reviewer can later answer: “Why did we choose this tool and why was this workflow allowed?”

18. Reference Sources Checked

TalkOps CI-Copilot — <https://github.com/talkops-ai/ci-copilot>

Architecture and feature reference for CI generation, policy, actionlint refinement and HITL. (CI-Copilot repository)

SanjaySundarMurthy pipeline-generator — <https://github.com/SanjaySundarMurthy/pipeline-generator>

Deterministic multi-platform pipeline generation reference. (pipeline-generator repository)

Open DevOps Agent — <https://github.com/open-devops-agent/open-devops-agent>

Grounded AI, plugin, audit and rollout patterns. (open-devops-agent repository)

skundu42 opendevops — <https://github.com/skundu42/opendevops>

Default-deny, argv-only execution, credential boundary and audit patterns. (opendevops repository)

Stakpak agent — <https://github.com/stakpak/agent>

Agent architecture and tool-boundary reference. (Stakpak agent repository)

JenkinsPipelineUnit — <https://github.com/jenkinsci/JenkinsPipelineUnit>

Jenkins pipeline testing. (JenkinsPipelineUnit repository)

Open Policy Agent — <https://github.com/open-policy-agent/opa>
Policy engine. (Open Policy Agent project documentation)

Syft — <https://github.com/anchore/syft>
SBOM generation. (Syft repository)

Cosign — <https://github.com/sigstore/cosign>
Artifact signing and attestations. (Cosign / Sigstore documentation)

Chainloop — <https://github.com/chainloop-dev/chainloop>
Supply-chain evidence store architecture. (Chainloop repository)

actionlint — <https://github.com/rhysd/actionlint>
GitHub Actions static checker. (actionlint project documentation)

Microsoft Foundry Responses API — <https://learn.microsoft.com/en-us/azure/foundry/openai/latest>
Structured Outputs and model API capabilities. (Microsoft Foundry Responses API documentation)

Microsoft Foundry Agent Service — <https://learn.microsoft.com/en-us/azure/ai-foundry/agents/overview>
Identity, RBAC, private networking and guardrails reference. (Microsoft Foundry Agent Service documentation)

GitHub Actions OIDC — <https://docs.github.com/en/actions/reference/security/oidc>
Least-privilege cloud authentication reference. (GitHub Actions OIDC documentation)

19. Final Recommendation

Start fresh, but do not start from zero. Use the current prototype only as a learning reference. Make the new system evidence-driven, policy-controlled, structured, testable and modular. Reuse OSS for specialized work, and make the AI planner/orchestrator, dynamic knowledge system, Pipeline IR, evidence model and governance workflow our core engineering contribution.

The most important implementation rule: whenever we are about to build a subsystem, first check whether a mature OSS package, CLI, SDK, protocol or service already solves it. If yes, integrate it unless there is a clear technical reason not to.